SESSION 2

Lists, Dictionaries, Functions & File Handling

Modern Data Engineering Course

Week 1 | Module 1: Programming Fundamentals

Instructors: Ameer Ul Islam

What You'll Learn Today

Lists

Ordered collections for storing sequences of data

Dictionaries

Key-value pairs — Python's most powerful data structure

Functions

Write reusable blocks of code to avoid repetition

File Handling

Read and write CSV, JSON, and text files

Lists: Ordered Collections

A list is like a spreadsheet column — an ordered sequence of items you can add to, remove from, and iterate over.

```
# Creating lists

tables = ["users", "orders", "products"]
row_counts = [5000, 12000, 3500]
mixed = ["sales", 42, True, 3.14]

# Accessing elements (0-indexed!)

tables[0]     # "users"
tables[-1]    # "products"
tables[0:2]   # ["users", "orders"]
len(tables)   # 3
```

Index Reference

```
[0]          [1]          [2]
"users"    "orders"
"products"
[-3]         [-2]
[-1]
```

Remember

Python counts from 0!
First element is index [0]

List Methods: Modify & Search

Adding & Removing

```
sources = ["api", "csv"]
# Add to end

sources.append("database")
# ["api", "csv", "database"]

# Insert at position
sources.insert(1, "webhook")


# Remove by value
sources.remove("csv")


# Remove last (returns it)
last = sources.pop()
```

Sorting & Searching

```
scores = [85, 92, 78, 95, 88]

scores.sort()

# [78, 85, 88, 92, 95]
scores.sort(reverse=True)

# [95, 92, 88, 85, 78]
min(scores)    # 78

max(scores)    # 95

sum(scores)    # 438


"api" in sources  # True
```

List Comprehensions

> A compact way to create lists — the Pythonic approach to transforming data.

Traditional Loop

```
# 4 lines of code
upper_tables = []
for t in tables:
    upper_tables.append(t.upper())
```

List Comprehension

```
# 1 line! Same result
upper_tables = [
    t.upper() for t in tables
]
```

```
# With condition (filter)
big_tables = [t for t in tables if row_counts[tables.index(t)] > 5000]
```

Tuples & Sets

Tuples — Immutable Lists

Like a list, but cannot be changed after creation. Used for fixed data.

```
# Creating tuples
coords = (23.8103, 90.4125)  # Dhaka
rgb = (255, 128, 0)
# Access like lists
print(coords[0])   # 23.8103
# Unpacking (very common!)
lat, lng = coords
```

Sets — Unique Values Only

Unordered collection with no duplicates. Great for deduplication.

```
# Creating sets
sources = {"api", "csv", "api", "db"}
print(sources) # {"api","csv","db"}
# Remove duplicates from list
ids = [1, 2, 3, 2, 1, 4]
unique = list(set(ids))
# [1, 2, 3, 4]
# Set operations
a = {1, 2, 3}
b = {2, 3, 4}
a & b  # {2, 3}  intersection
a | b  # {1,2,3,4} union
```

Dictionaries: Key-Value Pairs

Dictionaries are like a phone book — look up a value by its key. This is how JSON data works!

```
# Creating a dictionary

pipeline = {
    "name": "daily_sales_etl",
    "status": "running",
    "rows_processed": 15000,
    "is_active": True
}


# Accessing values

pipeline["name"]         # "daily_sales_etl"
pipeline.get("status")   # "running"
pipeline.get("owner", "unknown")
```

`dict["key"]`

Crashes with KeyError if key doesn't exist

`dict.get("key", default)`

Returns default value if key missing — much safer!

Dict Methods & Nested Structures

Common Methods

```
# Add or update
pipeline["owner"] = "Ameer"

# Delete a key
del pipeline["is_active"]

# Get all keys, values, items
pipeline.keys()
pipeline.values()
pipeline.items()  # key-value pairs
```

List of Dicts (Very Common!)

```
# Like rows in a table!

employees = [
  {"name": "Ali", "role": "DE"},
  {"name": "Sara", "role": "DA"},
  {"name": "Rafi", "role": "DE"},
]


employees[0]["name"]  # "Ali"
```

Looping Through Dictionaries

```
for key, value in pipeline.items():              # Loop key-value pairs
    print(f"{key}: {value}")

for emp in employees:                            # Loop list of dicts
    print(f"{emp['name']} is a {emp['role']}")
```

Break Time!

10 Minutes

Try creating a dictionary representing your favorite dataset

Functions: Reusable Code Blocks

> Functions let you write code once, use it many times. Like a recipe — define it once, cook it whenever you want.

```
def calculate_success_rate(total, failed):
    """Calculate pipeline success rate."""
    success = total - failed
    rate = (success / total) * 100
    return round(rate, 2)

# Call the function
result = calculate_success_rate(10000, 150)
print(result)  # 98.5
```

`def`
Keyword to define a function

`Parameters`
Input values (total, failed)

`Docstring`
"""Description""" — best practice

`return`
Send back a result

Function Parameters & Defaults

Default Parameters

```
def load_data(source, limit=1000,
              format="csv"):
    print(f"Loading {limit} rows")
    print(f"From {source} ({format})")

# All valid calls:
load_data("sales.csv")
load_data("orders", 500)
load_data("api", format="json")
load_data("db", 5000, "parquet")
```

Multiple Return Values

```
def analyze_data(data):
    total = len(data)
    avg = sum(data) / total
    return total, avg

# Unpack the results
count, average = analyze_data(
    [100, 200, 150, 300]
)
print(f"{count} items, avg {average}")
# 4 items, avg 187.5
```

File Handling: Reading & Writing

Reading Files

```
with open("data.txt", "r") as f:
    content = f.read()
    # file auto-closes!

# Read line by line
with open("log.txt") as f:
    for line in f:
        print(line.strip())
```

Writing Files

```
# Write (creates/overwrites)
with open("output.txt", "w") as f:
    f.write("Pipeline started\n")

# Append (adds to existing)
with open("output.txt", "a") as f:
    f.write("Complete!\n")
```

File Modes: "r" = read "w" = write (overwrites!) "a" = append "r+" = read + write

Always use 'with' — it automatically closes the file, even if errors occur!

Working with CSV Files

Reading CSV

```
import csv

with open("sales.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["product"])
        print(row["amount"])

# Each row is a dictionary:
# {"product": "Laptop",
#  "amount": "999.99"}
```

Writing CSV

```
import csv

data = [
  {"name": "Ali", "score": 95},
  {"name": "Sara", "score": 88},
]
with open("out.csv", "w",
          newline="") as f:
    writer = csv.DictWriter(
        f, fieldnames=["name","score"])
    writer.writeheader()
    writer.writerows(data)
```

Working with JSON Files

> JSON is the standard format for APIs. It maps directly to Python dicts and lists — very natural!

Reading JSON

```
import json

with open("config.json") as f:
    config = json.load(f)

print(config["database"])

# From string (API response)
data = json.loads(api_response)
```

Writing JSON

```
import json

result = {
    "pipeline": "sales_etl",
    "status": "success",
    "rows": 5000
}
with open("result.json", "w") as f:
    json.dump(result, f, indent=2)
```

Hands-On Exercise

Build a Student Grade Processor

1. Create a list of student dictionaries (name, scores list)
2. Write a function to calculate each student's average
3. Write a function to assign letter grades (A/B/C/D/F)
4. Loop through students, calculate grades
5. Write results to a CSV file
6. Save a JSON summary with class statistics

Hint: Use everything from today — lists, dicts, functions, csv module, json module!

Session 2 Summary

- Lists — ordered, mutable collections with powerful methods
- List comprehensions — concise data transformation
- Tuples — immutable sequences, great for fixed data
- Sets — unique values, deduplication
- Dictionaries — key-value pairs, the backbone of data
- Functions — def, parameters, return values
- File I/O — text, CSV, and JSON reading/writing
- Mini ETL — extract, transform, load pipeline

Up Next: Session 3

Pandas Fundamentals: DataFrames & Data Processing

- Introduction to Pandas DataFrames
- Reading data from CSV, Excel, JSON
- Selecting, filtering, and sorting data
- Basic aggregations and grouping

See you next session!